

MAT 242 Chapter 4 Runge-Kutta Fourth-Order Formula for 2×2 Systems

$$\mathbf{k}_1 = \mathbf{f}(t_n, \mathbf{x}_n), \mathbf{k}_2 = \mathbf{f}\left(t_n + \frac{1}{2}h, \mathbf{x}_n + \frac{1}{2}h\mathbf{k}_1\right), \mathbf{k}_3 = \mathbf{f}\left(t_n + \frac{1}{2}h, \mathbf{x}_n + \frac{1}{2}h\mathbf{k}_2\right), \mathbf{k}_4 = \mathbf{f}(t_n + h, \mathbf{x}_n + h\mathbf{k}_3)$$

$$\mathbf{x}_{n+1} = \mathbf{x}_n + \frac{h}{6}(\mathbf{k}_1 + 2\mathbf{k}_2 + 2\mathbf{k}_3 + \mathbf{k}_4)$$

$$\text{where } \mathbf{x} = \begin{bmatrix} x(t) \\ y(t) \end{bmatrix} \text{ and } \mathbf{x}'(t) = \mathbf{f}(t, \mathbf{x}) = \begin{bmatrix} x'(t) \\ y'(t) \end{bmatrix} \text{ and } \mathbf{k}_i = \begin{bmatrix} F_i \\ G_i \end{bmatrix}.$$

Hector Martinez — June 18, 2026

We are given the following conditions:

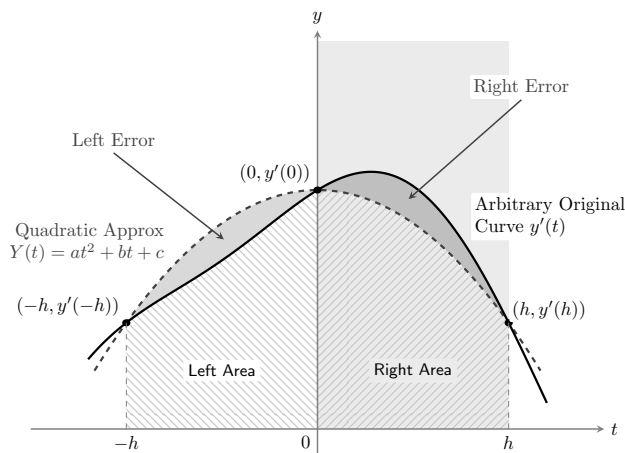
$$\begin{aligned} x' &= -x + y - (1 + t^3)e^{-t}, & x(0) &= 0 \\ y' &= -x - y - (t - 3t^2)e^{-t}, & y(0) &= 1 \\ x(t) &= e^{-t}(\sin(t) - t), & y(t) &= e^{-t}(\cos(t) + t^3). \end{aligned}$$

Our goal is to use the **Runge-Kutta Fourth-Order** method to approximate the values $x(1)$ and $y(1)$ using a step size of $h = 0.05$ to five decimals.

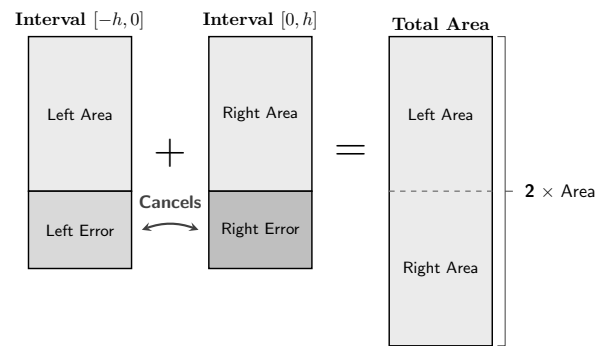
Isolating the y -component for the purpose of the derivation, we can apply the Fundamental Theorem of Calculus to its derivative:

$$\begin{aligned} y(t_{n+1}) - y(t_n) &= \int_{t_n}^{t_{n+1}} y'(t) dt \\ \implies y(t_{n+1}) &= \int_{t_n}^{t_{n+1}} y'(t) dt + y(t_n). \end{aligned}$$

To reduce error, our goal is to approximate the function inside the integral with a second-degree polynomial of the form $at^2 + bt + c$, making it simpler to integrate. To do this, we must first decide where to center our interval. Centering it at the step's actual midpoint $\frac{h}{2}$ would be ideal conceptually, but it increases the algebraic overhead significantly; however, centering our symmetric window at 0 over the interval $[-h, h]$ allows us to exploit symmetry and induce term cancellations. Because the window is symmetric about the origin, the local over-guesses on one half balance out the under-guesses on the other half, yielding a cancellation that significantly increases the accuracy of our forward step. You will notice, however, that the result of this integration over $[-h, h]$ yields $2 \times \text{Area}$ of our actual forward step. This simply means we will scale our final result by half later on to keep it accurate to our actual interval of $[0, h]$.



Quadratic Approximation



Numerical Area Cancellation

To begin, we assume our approximating curve, which we will denote as $Y(t)$, to fit the form $at^2 + bt + c$. This implies that the general integral of $Y(t)$ over our symmetric interval should abide by the form:

$$\begin{aligned}\int_{-h}^h (at^2 + bt + c) dt &= \left[\frac{a}{3}t^3 + \frac{b}{2}t^2 + ct \right]_{-h}^h \\ &= \frac{2}{3}ah^3 + 2ch.\end{aligned}$$

Using this information about the general shape of our integral, we can enforce a set of coordinate constraints to narrow down the possibilities for $Y(t)$:

$$\begin{aligned}\text{At } t = -h : \quad &a(-h)^2 + b(-h) + c = y'_{-h} \\ \text{At } t = 0 : \quad &c = y'_0 \\ \text{At } t = h : \quad &a(h)^2 + b(h) + c = y'_h.\end{aligned}$$

Where y'_{-h} , y'_0 , and y'_h represent our local quadratic approximation evaluated at their respective values of t : $-h$, 0 , and h .

Because we know $c = y'_0$, we can make the substitution:

$$\begin{aligned}\text{At } t = -h : \quad &a(-h)^2 + b(-h) + y'_0 = y'_{-h} \\ \text{At } t = h : \quad &a(h)^2 + b(h) + y'_0 = y'_h.\end{aligned}$$

This is where the symmetry comes in to aid us. Since b is being multiplied by a negative in the first equation, summing both equations will cause that term to cancel, yielding our final coordinate constraint:

$$\begin{aligned}[a(-h)^2 + b(-h) + y'_0] + [a(h)^2 + b(h) + y'_0] &= 2ah^2 + 2y'_0 \\ &= y'_h + y'_{-h} \\ \implies \text{constraint: } 2ah^2 &= y'_{-h} - 2y'_0 + y'_h.\end{aligned}$$

This establishes the exact rules our approximating curve, $Y(t)$, must follow to be a valid approximation. Given this set of constraints, we can now substitute them back into our general integrated result:

$$\begin{aligned}\int_{-h}^h (at^2 + bt + c) dt &= \frac{2}{3}ah^3 + 2ch \\ &= \frac{h}{3} (2ah^2 + 6c) \\ &= \frac{h}{3} [(y'_{-h} - 2y'_0 + y'_h) + 6y'_0] \\ &= \frac{h}{3} (y'_{-h} + 4y'_0 + y'_h).\end{aligned}$$

To move forward, and for simplicity, we will denote our initial slope as:

$$k_1 = y'_{-h} = y'(t_n, y_n).$$

From here, we can approximate the slope at the midpoint, which we will denote as k_2 :

$$k_2 = y' \left(t_n + \frac{h}{2}, y_n + \frac{h}{2}k_1 \right).$$

It is crucial that we pause here before defining our endpoint value y'_h . Extrapolating to the end of the interval using only our initial slope assumes that the rate of change remains constant past the midpoint, which can introduce error if the derivative changes significantly.

To circumvent this, we sample a preparatory midpoint slope using k_2 , allowing us to re-evaluate our trajectory in case of a major change. This updated midpoint slope is denoted as k_3 :

$$k_3 = y' \left(t_n + \frac{h}{2}, y_n + \frac{h}{2}k_2 \right).$$

With a more accurate representation of the slope crossing the midpoint, we can extrapolate forward to approximate our endpoint slope, denoted as k_4 :

$$k_4 = y'_h = y'(t_n + h, y_n + hk_3).$$

Because the midpoint slope y'_0 is represented by two distinct approximations (k_2 and k_3), we generalize y'_0 by taking their average:

$$y'_0 \approx \frac{k_2 + k_3}{2}.$$

Now, substituting our boundary values $y'_{-h} = k_1$ and $y'_h = k_4$, along with our averaged midpoint approximation, directly into our integrated formula yields:

$$\begin{aligned} \int_{-h}^h y'(t) dt &\approx \frac{h}{3} (y'_{-h} + 4y'_0 + y'_h) \\ &\approx \frac{h}{3} \left(k_1 + 4 \left[\frac{k_2 + k_3}{2} \right] + k_4 \right) \\ &\approx \frac{h}{3} (k_1 + 2k_2 + 2k_3 + k_4). \end{aligned}$$

This equation, in its current state, represents the analytical solution to integrating $Y(t)$ over $[-h, h]$. However, as stated near the beginning, this area is exactly double the area from 0 to h , our forward step. Because of this, we must halve our result to isolate the solution over $[0, h]$, resulting in:

$$\Delta y \approx \frac{h}{6} (k_1 + 2k_2 + 2k_3 + k_4).$$

By substituting our definitions for k_1 , k_2 , k_3 , and k_4 , we can expand the system to evaluate our generalized value for $y(t_{n+1})$ numerically. Evaluating explicitly at our first discrete step where $t_n = 0$:

$$\begin{aligned} y(t_1) = y_0 + \frac{h}{6} &\left[-1 + 2 \left(\frac{h}{2} - 1 - \left(\frac{h}{2} - \frac{3h^2}{4} \right) e^{-h/2} \right) \right. \\ &+ 2 \left(-\frac{h}{2} \left[1 - \frac{h}{2} - \left(1 + \frac{h^3}{8} \right) e^{-h/2} \right] - \left(\frac{h}{2} - \frac{3h^2}{4} \right) e^{-h/2} \right. \\ &\quad \left. \left. - \left[1 + \frac{h}{2} \left(\frac{h}{2} - 1 - \left(\frac{h}{2} - \frac{3h^2}{4} \right) e^{-h/2} \right) \right] \right) \right. \\ &+ \left(-h \left[-\frac{h}{2} \left(1 - \frac{h}{2} - \left(1 + \frac{h^3}{8} \right) e^{-h/2} \right) \right. \right. \\ &\quad \left. \left. + \left(1 + \frac{h}{2} \left[\frac{h}{2} - 1 - \left(\frac{h}{2} - \frac{3h^2}{4} \right) e^{-h/2} \right] \right) - \left(1 + \frac{h^3}{8} \right) e^{-h/2} \right] \right. \right. \\ &\quad \left. \left. - \left[1 + h \left(-\frac{h}{2} \left(1 - \frac{h}{2} - \left(1 + \frac{h^3}{8} \right) e^{-h/2} \right) \right. \right. \right. \right. \\ &\quad \left. \left. \left. - \left[1 + \frac{h}{2} \left(\frac{h}{2} - 1 - \left(\frac{h}{2} - \frac{3h^2}{4} \right) e^{-h/2} \right) \right] - \left(\frac{h}{2} - \frac{3h^2}{4} \right) e^{-h/2} \right) \right] \right. \right. \\ &\quad \left. \left. - (h - 3h^2)e^{-h} \right) \right] \\ &\approx 0.95016. \end{aligned}$$

Because our analytical solution was derived for any general quadratic function, the only difference in the numerical solution between x and y is in their slopes: k_1 , k_2 , k_3 , and k_4 . Substituting $x'(t)$ into our analytical formula yields

the secondary evaluation for $x(t_1)$:

$$\begin{aligned}
x(t_1) = x_0 + \frac{h}{6} & \left[0 + 2 \left(1 - \frac{h}{2} - \left(1 + \frac{h^3}{8} \right) e^{-h/2} \right) \right. \\
& + 2 \left(-\frac{h}{2} \left[1 - \frac{h}{2} - \left(1 + \frac{h^3}{8} \right) e^{-h/2} \right] - \left(1 + \frac{h^3}{8} \right) e^{-h/2} \right. \\
& \quad \left. \left. + \left[1 + \frac{h}{2} \left(-\left(1 - \frac{h}{2} \right) - \left(\frac{h}{2} - \frac{3h^2}{4} \right) e^{-h/2} \right] \right) \right) \right. \\
& + \left(-h \left[-\frac{h}{2} \left(1 - \frac{h}{2} - \left(1 + \frac{h^3}{8} \right) e^{-h/2} \right) - \left(1 + \frac{h^3}{8} \right) e^{-h/2} \right. \right. \\
& \quad \left. \left. + \left(1 + \frac{h}{2} \left[-\left(1 - \frac{h}{2} \right) - \left(\frac{h}{2} - \frac{3h^2}{4} \right) e^{-h/2} \right] \right) \right] \right) \right. \\
& + \left[1 + h \left(-\frac{h}{2} \left[1 - \frac{h}{2} - \left(1 + \frac{h^3}{8} \right) e^{-h/2} \right] \right. \right. \\
& \quad \left. \left. - \left[1 + \frac{h}{2} \left(\frac{h}{2} - 1 - \left(\frac{h}{2} - \frac{3h^2}{4} \right) e^{-h/2} \right) \right] - \left(\frac{h}{2} - \frac{3h^2}{4} \right) e^{-h/2} \right) \right] \\
& \quad \left. \left. - (1 + h^3) e^{-h} \right) \right] \\
& \approx -0.00002.
\end{aligned}$$

These represent the first step toward $y(t_n)$ and $x(t_n)$. In order to fully cover t_0 to t_n , we must run this approximation n more times. To figure out what the value of n is, we can relate it to h via the identity:

$$\begin{aligned}
h &= \frac{t_{\text{final}} - t_0}{n} \\
\Rightarrow n &= \frac{t_{\text{final}} - t_0}{h} \\
&= \frac{1 - 0}{0.05} = 20.
\end{aligned}$$

Therefore, we must run this process 19 more times to reach our final approximations for $y(1)$ and $x(1)$. Table 1 presents the computed x_i and y_i values for each index.

Table 1: Computed x_i and y_i Values

i	0	1	2	3	4	5	6	7	8	9	10
t_i	0.00	0.05	0.10	0.15	0.20	0.25	0.30	0.35	0.40	0.45	0.50
x_i	0.00000	-0.00002	-0.00015	-0.00048	-0.00109	-0.00202	-0.00332	-0.00500	-0.00709	-0.00959	-0.01248
y_i	1.00000	0.95016	0.90122	0.85395	0.80896	0.76676	0.72773	0.69218	0.66031	0.63225	0.60810

i	11	12	13	14	15	16	17	18	19	20
t_i	0.55	0.60	0.65	0.70	0.75	0.80	0.85	0.90	0.95	1.00
x_i	-0.01576	-0.01940	-0.02339	-0.02770	-0.03229	-0.03713	-0.04219	-0.04744	-0.05282	-0.05832
y_i	0.58785	0.57150	0.55896	0.55014	0.54490	0.54311	0.54457	0.54912	0.55654	0.56665

The values presented in Table 1 represent the final numerical approximations of $x(t_i)$ and $y(t_i)$ at each index of the summation. Taking our values at the final step where $i = 20$ and $t = 1$, we achieve our final approximations for the system:

$$\begin{aligned}
x(1) &\approx -0.05832 \\
y(1) &\approx 0.56665
\end{aligned}$$

While a formal proof of fourth-order accuracy could be made, running through a full error analysis lies beyond the scope of this paper.